

CyberLens 2.0 — Explained in Plain English

What is CyberLens 2.0?

Imagine you're the Chief Risk Officer of a big Indian bank. Your boss asks you: "How much cyber risk are we actually exposed to, and what should we spend on security?"

Right now, all you have is vague reports saying "we're at medium risk" or "we need to improve security." That's like saying "we might get robbed" without knowing how much cash you have, where the weak points are, or which locks to buy.

CyberLens 2.0 solves this. It takes your actual assets (servers, apps, databases), your known vulnerabilities, and your transaction volumes — and turns it all into concrete numbers:

- "You're exposed to ₹4,200 crores of potential losses per year."
 - "Your security score is 38 out of 100."
 - "You should spend ₹50 lakhs on these 3 controls to reduce your risk by ₹900 crores."
-

The Data Layer: What It Stores

Think of this as the "brain's memory" — everything CyberLens knows about your bank.

Assets (What You're Protecting)

Each asset is a real thing in your bank:

Asset	What It Is	Why It Matters
NPCI UPI Switch	The system that processes UPI payments	5 million transactions/day, costs ₹25 lakhs/hr if it goes down
Core Banking Server	The main database with all customer accounts	1 million transactions/day, costs ₹50 lakhs/hr if it goes down
Mobile Banking App	The app on your phone	2 million transactions/day
ATM Network	All your ATMs connected together	500,000 transactions/day

Each asset has: - **Criticality:** 1-10 scale (10 = if this fails, the bank fails) - **Daily Transaction Volume:** How many transactions flow through it - **Downtime Cost Per Hour:** How much money you lose every hour it's down - **Crypto Profile:** What encryption it uses (RSA-2048, ECC-224, etc.)

Example:

```
UPI Switch
├─ Criticality: 9/10
├─ Transactions: 5,000,000/day
├─ Downtime Cost: ₹25,00,000/hr (₹25 lakhs)
└─ Crypto: RSA-2048
```

Vulnerabilities (The Weak Points)

These are known security flaws in your systems:

Vuln ID	CVE	CVSS	Days Unpatched	Exploit Available	Category
V_UPI_001	CVE-2024-1234	8.5	12 days	✓ Yes	Authentication Bypass
V_CBS_001	CVE-2024-3456	9.1	8 days	✓ Yes	SQL Injection
V_MOBILE_001	CVE-2024-5678	7.8	15 days	✓ Yes	Insecure Storage

What each field means: - **CVSS:** How severe the vulnerability is (0-10, higher = worse) - **Days Unpatched:** How long the flaw has been sitting there unfixed - **Exploit Available:** Are hackers already using this? (Yes = more dangerous) - **Category:** What type of attack it enables (SQL injection, authentication bypass, etc.)

RBI Mappings (The Rules You're Breaking)

This is where CyberLens ties vulnerabilities to actual RBI/SEBI/NPCI compliance requirements:

```
Vulnerability: Authentication Bypass on UPI Switch
↓
Maps to: RBI/2023-24/105.A.3
↓
Rule: "Mandatory multi-factor authentication for all payment transactions above ₹50,000"
↓
Violation: You're breaking this RBI circular
```

This is huge for Indian banks — you're not just saying "we're vulnerable," you're saying "we're violating RBI circular XYZ, section ABC."

The Risk Engine: How It Calculates Everything

This is the core logic — where all the magic happens.

1. RBI-Weighted CVSS (Making It India-Specific)

Standard CVSS (a global scoring system) isn't enough for India. CyberLens adjusts it based on:

Formula:

```
RBI-Weighted CVSS = Base CVSS
                    × Asset Criticality (1-10 / 10)
                    × RBI Multiplier (1.0-1.6 depending on asset type)
                    × Patch Penalty (more days unpatched = higher penalty)
                    × Exploit Boost (1.5x if exploit is available)
```

Example Calculation:

```
Vulnerability: Authentication Bypass on UPI Switch
├─ Base CVSS: 8.5
├─ Asset Criticality: 9/10 = 0.9
├─ RBI Multiplier (UPI Switch + Auth Bypass): 1.5
├─ Days Unpatched: 12 → Patch Penalty: 12/7 = 1.71
├─ Exploit Available: Yes → Boost: 1.5

RBI-Weighted CVSS = 8.5 × 0.9 × 1.5 × (1 + 1.71/10) × 1.5
                  = 8.5 × 0.9 × 1.5 × 1.171 × 1.5
                  = 9.2 (capped at 10)
```

Why this matters: - A CVSS 8.5 on a critical UPI switch with an available exploit is way worse than a CVSS 8.5 on a low-criticality logging server - The patch penalty means the longer you wait, the worse your score gets - RBI multipliers ensure banking-critical systems are scored higher

2. Cyber Resilience Index (CR-I): Your Security Grade

This is your overall security score (0-100). Think of it like a credit score for cyber risk.

Formula:

```
CR-I = 100 - (Weighted Average of Vulnerability Impacts)

Where:
- Each vulnerability's impact = (10 - RBI-Weighted CVSS) × Asset Risk Weight
- Asset Risk Weight = Criticality × Daily Transactions × Quantum Risk Multiplier
```

Example:

```
UPI Switch has 5 vulnerabilities:
├─ Vuln 1: RBI-CVSS = 9.2 → Impact = (10 - 9.2) × Weight = 0.8 × Weight
├─ Vuln 2: RBI-CVSS = 7.5 → Impact = (10 - 7.5) × Weight = 2.5 × Weight
├─ Vuln 3: RBI-CVSS = 6.8 → Impact = (10 - 6.8) × Weight = 3.2 × Weight
└─ ...

CR-I = 100 - Average Impact
     = 38.5 (out of 100)
```

What the score means: - **0-30:** Critical risk — you need immediate action - **31-50:** High risk — serious vulnerabilities need fixing - **51-70:** Moderate risk — room for improvement - **71-90:** Low risk — solid security posture - **91-100:** Excellent — well-protected

Quantum Risk Multiplier: If you're using weak encryption (RSA-512, 3DES, SHA-1), your risk goes up 20% because quantum computers (when they arrive) will break these easily.

3. Expected Annual Loss (EAL): The ₹ Number

This is the big one — it turns technical risk into **money you could lose**.

Formula:

$$\text{EAL} = \text{Probability of Attack} \times \text{Loss Per Incident}$$

Where:

- Probability = RBI-Weighted CVSS / 10 (capped at 0.9)
- Loss Per Incident = (Downtime Cost × 6 hours) + (Daily Transactions × 0.01% fraud rate)

Example:

Vulnerability: Authentication Bypass on UPI Switch

- RBI-Weighted CVSS: 9.2
- Probability of Attack: $9.2/10 = 0.92$ (capped at 0.9 = 90%)
- Downtime Cost: ₹25,00,000/hr × 6 hrs = ₹1,50,00,000
- Fraud Loss: 5,000,000 transactions × 0.01% = ₹5,00,000
- Loss Per Incident: ₹1,55,00,000

$$\begin{aligned} \text{EAL} &= 0.9 \times ₹1,55,00,000 \\ &= ₹1,39,50,000 \text{ per year (₹1.4 crores)} \end{aligned}$$

Total Exposure:

Sum of EAL across ALL vulnerabilities on ALL assets
= ₹4,200 crores per year (for the demo scenario)

This is the number that makes executives sit up and pay attention. It's not "we might get hacked" — it's "we could lose ₹4,200 crores this year."

The Controls Library: What You Can Buy to Fix Things

CyberLens doesn't just tell you what's wrong — it tells you what to do about it.

Security Controls (The Remedies)

Each control is a security measure you can implement:

Control	Cost (₹)	What It Does	Effectiveness
MFA for UPI	₹80 lakhs	Multi-factor auth for high-value transactions	75%
Web Application Firewall	₹40 lakhs	Blocks SQL injection, XSS attacks	60%
Accelerated Patching	₹20 lakhs	Patch critical vulns within 7 days	80%
System Hardening	₹30 lakhs	Hardware keystores, AES-256 encryption	70%
EDR Monitoring	₹50 lakhs	Endpoint detection, 24/7 SOC	55%
Rate Limiting	₹60 lakhs	Transaction velocity checks, anomaly detection	85%
Penetration Testing	₹15 lakhs	Annual security audits	45%
Encrypted Backups	₹70 lakhs	Immutable backups, DR playbooks	65%

How Effectiveness Works: If you deploy the WAF (60% effective) against SQL injection vulnerabilities:

```
Before WAF: RBI-Weighted CVSS = 9.1
After WAF: 9.1 × (1 - 0.60) = 9.1 × 0.40 = 3.64

Vulnerability reduced from Critical to Low!
```

Controls affect different vulnerability types: - MFA → Authentication Bypass - WAF → SQL Injection, XSS, Injection - Patching → Everything (it's the silver bullet) - EDR → Privilege Escalation, Network Sniffing - Rate Limiting → Rate Limiting issues - Backups → Ransomware, Data Exposure

The Budget Optimizer: How to Spend Your Money

This is where CyberLens becomes a **financial advisor for security**.

ROSI (Return on Security Investment)

For each control, CyberLens calculates:

```
ROSI = (Risk Reduction - Cost) / Cost × 100

Example: WAF Deployment
├─ Cost: ₹40 lakhs
├─ Risk Reduction: ₹400 crores/year (reduces multiple vulns)
├─ ROSI = (₹400 Cr - ₹40 L) / ₹40 L × 100
      = 1600% return!

For every ₹1 you spend, you reduce ₹16 of risk.
```

Greedy Knapsack Algorithm (The Optimizer)

With a budget of ₹1 crore, CyberLens picks controls that maximize risk reduction:

Step 1: Rank all controls by ROSI (best to worst)

— 1. Patch Management (2000% ROSI) - ₹20L ✓ Fits in budget

— 2. Rate Limiting (1200% ROSI) - ₹60L ✓ Fits in budget

— 3. WAF (1000% ROSI) - ₹40L ✗ Exceeds budget

— 4. MFA (500% ROSI) - ₹80L ✗ Exceeds budget

Step 2: Select controls until budget is exhausted

— Selected: Patch Management (₹20L) + Rate Limiting (₹60L)

— Total Cost: ₹80L

— Remaining Budget: ₹20L

— Total Risk Reduction: ₹700 crores/year

Output Table:

Control	Cost (₹)	Risk Reduction	ROSI
Patch Management	₹20L	₹300 Cr/yr	2000%
Rate Limiting	₹60L	₹400 Cr/yr	1200%
Total	₹80L	₹700 Cr/yr	1800%
Remaining Budget	₹20L		

Before vs. After:

Before: ₹4,200 crores exposure

After: ₹3,500 crores exposure (₹700 Cr reduction)

Saved: ₹700 crores for just ₹80 lakhs investment!

The LLM Remediation: AI-Generated Fixes

CyberLens has a built-in AI (TinyLlama, quantized to run locally) that generates simple remediation steps.

When you click "Show Remediation" on a vulnerability:

Input:

— Vulnerability: Authentication Bypass

— Asset: UPI Switch

— CVE: CVE-2024-1234

— RBI Clause: RBI/2023-24/105.A.3

AI Output (Hindi/English):

- "Admin panel पर SMS-OTP अनिवार्य करें।"
(Enable SMS-OTP on admin panel)
- "Enable rate limiting on login API endpoint."
- "Add IP whitelist for admin access."

Why this matters: - Non-technical branch staff can understand what to do - Hindi instructions for rural bank branches - If AI fails (no GPU, model not loaded), it falls back to template-based fixes

The UI: What You See and Interact With

Executive View (For Bank Management)

What they see:



What they can do: 1. Load demo scenario → See realistic numbers 2. Adjust budget slider → See which controls to buy 3. Click "Generate SIH Summary" → Get submission file for hackathon 4. Click "View Details" → Jump to technical view for that asset

Technical View (For Security Engineers)

What they see:

Technical View

Select Asset: [NPCI UPI Switch ▼] Search: [CVE-2024...] |

NPCI UPI Switch [Database]
Criticality: 9/10 · Txns: 5M/day · Downtime: ₹25L/hr
Resilience: 38.5/100 Exposure: ₹1,800 Cr/yr

Vulnerabilities

Vuln ID	CVE	CVSS	RBI-Wtd	Days	Exploit	Risk
V_UPI_001	CVE-2024-X	8.5	9.2	12	✓	₹800Cr
V_UPI_002	CVE-2024-Y	7.2	7.8	25	✗	₹300Cr

[Show Details] [Simulate Fix]

(When you click "Show Details" on V_UPI_001)

Technical Description: Unauthorized access to admin panel...

RBI Compliance [Expand ▼]

✓ RBI/2023-24/105.A.3 - Authentication Security

✓ SEBI/HO/ISD/CIR/P/2020/168 - Access Control

AI-Generated Remediation (Hindi/English)

1. Admin panel पर SMS-OTP अनिवार्य करें।

2. Enable rate limiting on login API.

3. Add IP whitelist for admin access.

What-If Simulator

☒ WAF Deployment

Cost: ₹25L

Effectiveness: 60%

☒ Rate Limiting

Cost: ₹15L

Effectiveness: 30%

Current Risk: ₹800 Cr/yr New Risk: ₹320 Cr/yr (Δ -60%)

Updated CR-I: 38.5 → 62.1

What they can do: 1. Select specific asset → See its vulnerabilities 2. Search by CVE ID → Find specific vulnerabilities 3. Click "Show Details" → See technical info, RBI mapping, AI remediation 4. Toggle what-if switches → See real-time risk reduction 5. Click "Simulate Fix" → See most effective control for that vuln 6. View compliance heatmap → See which RBI rules are violated

The What-If Simulator: Testing Before Buying

This is like a flight simulator for security investments.

Before:

Vulnerability: Authentication Bypass on UPI Switch

- Current Risk: ₹800 crores/year
- CR-I: 38.5/100
- Controls Applied: None

You toggle switches:

☒ WAF Deployment (60% effective)
☒ Rate Limiting (30% effective)
☐ IP Whitelist (10% effective)

After:

$$\begin{aligned} \text{New RBI-Weighted CVSS} &= 9.2 \times (1 - 0.60) \times (1 - 0.30) \\ &= 9.2 \times 0.40 \times 0.70 \\ &= 2.58 \end{aligned}$$

New Risk = ₹256 crores/year (down from ₹800 Cr)
New CR-I = 62.1/100 (up from 38.5)

Why this is powerful: - Test controls before spending actual budget - See cumulative effect (multiple controls stack) - Make data-driven decisions - Avoid wasting money on ineffective controls

The Compliance Heatmap: Where You're Violating Rules

A visual grid showing which RBI rules are violated across your assets:

	UPI Switch	Core DB	Payment GW
Authentication	45%	95%	78%
Patch Management	60%	30%	92%
Encryption	100%	100%	100%
Network Segmentation	75%	88%	55%

Color coding: -

Green (≥90%): Compliant

 -

Yellow (60-89%): Partially compliant

 -

Red (<60%): Non-compliant

This helps you: - See at a glance where you're violating RBI rules - Prioritize which assets need the most attention - Show auditors exactly where you're compliant/non-compliant

The Deployment: How to Run It

Zero-Setup Demo (Docker)

```
docker compose up --build
```

This: 1. Builds a Docker image with all dependencies 2. Launches Streamlit app on port 8501 3. Loads sample data (fake but realistic Indian banking data) 4. Ready to demo in 2 minutes

No API keys needed. No cloud accounts. No complex setup. Just one command.

Direct Installation

```
pip install -r requirements.txt  
streamlit run streamlit_app.py
```

Optional: FastAPI Backend

If you want to expose the risk engine as a REST API:

```
ENABLE_API=true docker compose up --build
```

Then you can call it like:

```
GET /assets → List all assets  
POST /cr-i → Calculate Cyber Resilience Index  
POST /optimize/budget → Get optimal control selection  
POST /llm/remediate → Get AI-generated remediation steps
```

The SIH (Smart India Hackathon) Features

This is built specifically for the hackathon:

1. Load UPI Switch Demo Scenario

One click loads a realistic Indian banking scenario with: - 9 assets (UPI Switch, Core Banking, Mobile App, etc.) - 10 vulnerabilities (real CVEs mapped to Indian banking context) - RBI compliance mappings

2. Generate SIH Summary

Click this button and get a markdown file with: - Problem statement - Solution overview - Key metrics (₹ protected, users safeguarded) - Technical architecture - Future scope

Upload this file directly to the SIH portal.

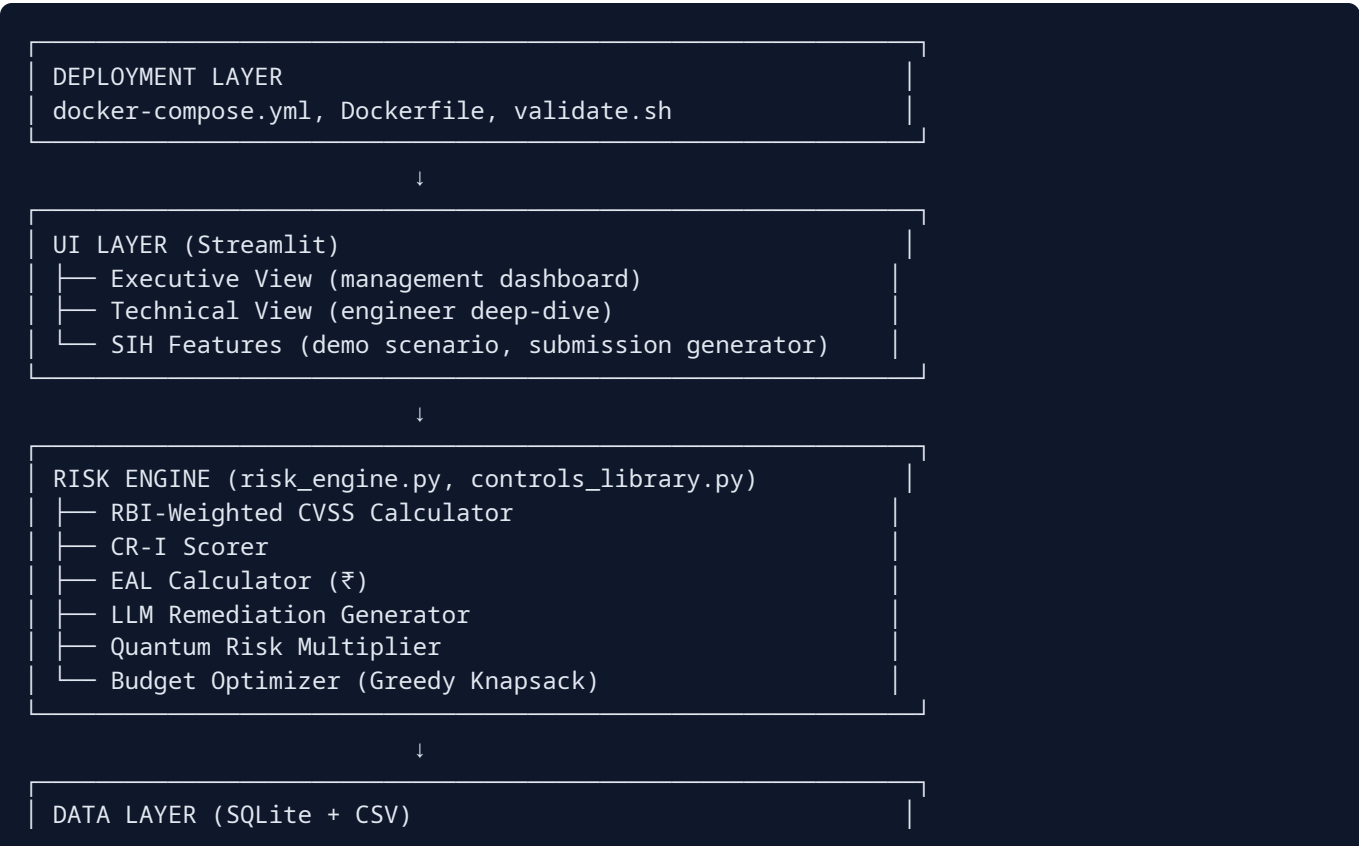
The Audit Trail: For RBI Compliance

Every risk calculation is logged in `audit_log.jsonl`:

```
{
  "timestamp": 1694371200,
  "function": "compute_risk_matrix",
  "vuln_rows": 10,
  "assets": 9
}
```

Why this matters: - RBI auditors can see exactly how you calculated risk - Transparent, reproducible calculations - No black boxes

The Architecture: How It All Fits Together



- Assets (what you're protecting)
- Vulnerabilities (weak points)
- Incidents (historical breaches)
- RBI Mappings (compliance rules)

Data Flow: 1. Startup → Load CSV data into SQLite 2. User interacts → Dashboard queries data 3. Risk calculations → Engine processes vulnerabilities 4. Optimizer runs → Recommends controls 5. What-if simulation → Recalculates in real-time 6. LLM generates → Remediation steps 7. SIH submission → Generates markdown file

Future Work: What Could Be Added

Real-Time Feeds

- CERT-In vulnerability alerts (automatic updates)
- NVD (National Vulnerability Database) integration
- SIEM log ingestion

ML-Enhanced Scoring

- Replace rule-based scoring with XGBoost model
- Train on real Indian banking breach data
- Predict likelihood of attack based on patterns

Exact Optimization

- Use PuLP (linear programming) instead of greedy algorithm
- Find mathematically optimal control selection
- Handle complex constraints (e.g., "must buy control A before B")

Cloud-Native Deployment

- Kubernetes Helm chart
- Microservices architecture (FastAPI + React)
- Horizontal scaling for enterprise use

Multi-Language Support

- Hindi/English toggle throughout
- Regional language support (Tamil, Telugu, Bengali, etc.)

Lightweight Rural Build

- Stripped-down version for Raspberry Pi
- Deploy in rural bank branches with limited internet
- Offline-first design

Automated Reporting

- PDF/Excel reports for RBI inspections
- Automated compliance documentation
- Executive summaries with charts

Key Metrics to Remember

Metric	What It Means	Demo Value
Total Exposure	Annual ₹ at risk	₹4,200 Cr
CR-I	Security score (0-100)	38/100
RBI Compliance	% of compliant assets	58%
ROSI	Return on security investment	1800% avg
EAL per Vuln	Expected annual loss per vulnerability	₹1.4 Cr - ₹800 Cr
Optimal Plan	Best controls to buy	₹80L cost, ₹700 Cr reduction

Why This Matters for Indian Banking

1. **RBI Compliance:** Ties vulnerabilities to specific RBI circulars (not just generic CVSS)
2. **₹-Denominated:** Financial impact in crores, not vague "high risk" labels
3. **UPI-Specific:** Designed for India's digital payment infrastructure
4. **Hindi Remediation:** AI generates fixes in Hindi for rural staff
5. **Budget Optimization:** Tells CFOs exactly where to spend
6. **Audit Trail:** Transparent calculations for RBI inspectors
7. **Zero-Setup Demo:** Ready to show in 2 minutes

Bottom line: CyberLens 2.0 turns "we might get hacked" into "we're exposed to ₹4,200 crores of risk, here's exactly what to buy with our ₹1 crore budget, and here's how much safer we'll be."

It's not just a security tool — it's a **financial planning tool for cyber risk**.